Database Fundamentals

Capstone Project

ERD · Mapping · Normalization · SQL DML · Aggregation · Joins · Subqueries · Views · Stored Procedures · Triggers · Scheduler Jobs

Name: Al-Jolanda Al-Handhali

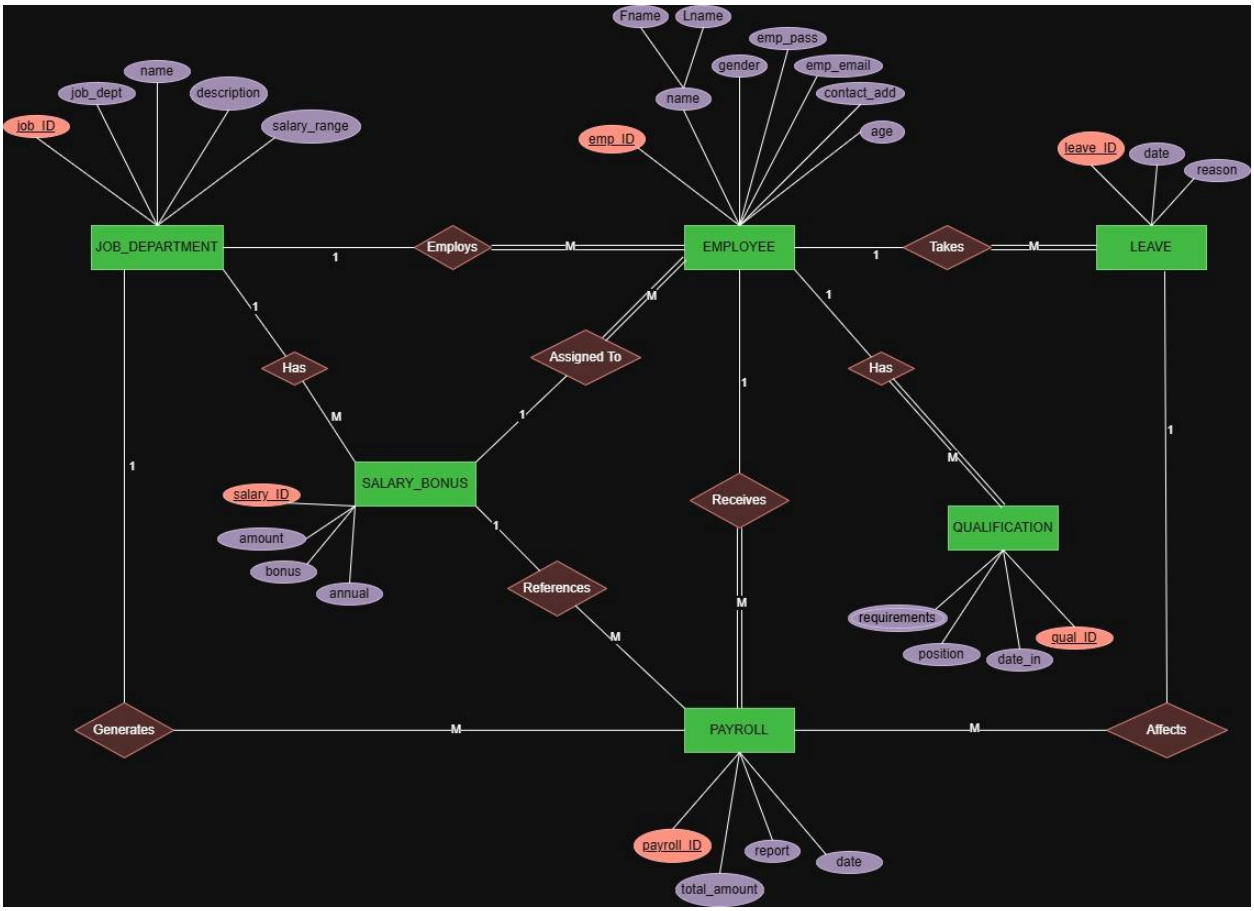
Date: 20 June 2026

Contents

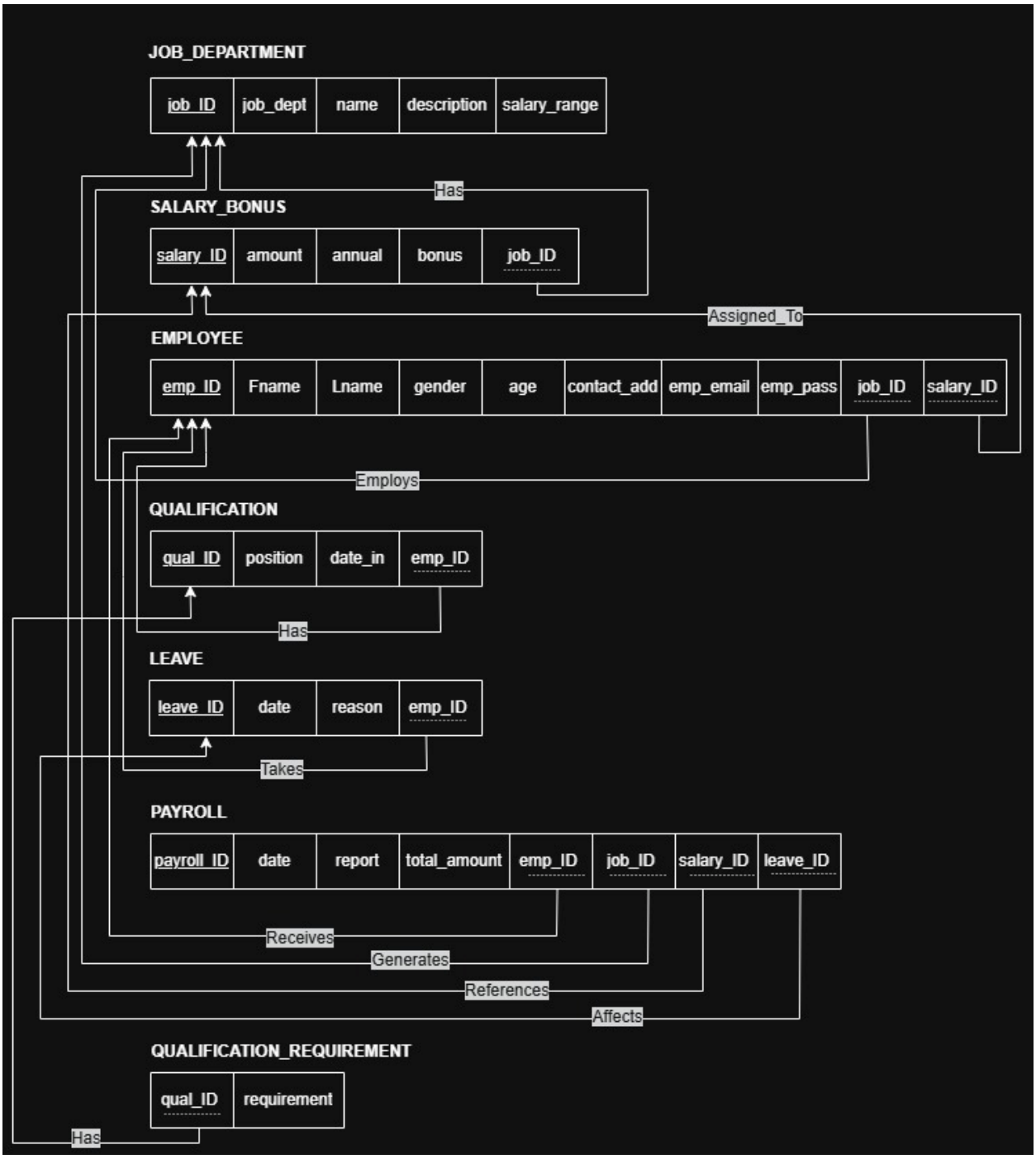
1. ERD Design & Mapping 3
Task 1: Draw the Full ERD 3
Task 2: Map the ERD to a Relational Schema 4
Task 3: Write the Full DDL CREATE Statements 5
2. Normalization 7
Task 1: First Normal Form (1NF) Analysis 7
Task 2: Second Normal Form (2NF) Analysis 7
Task 3: Third Normal Form (3NF) Analysis 8
3. SQL DML — INSERT / UPDATE / DELETE / SELECT 9
Task 1: Populate All Tables with Seed Data 9
Task 2: Conditional SELECT Queries 10
4. Aggregation Functions 12
Task 1: Basic Aggregation 12
Task 2: GROUP BY with HAVING 13
5. Joins 14
Task 1: INNER JOIN — Employee Full Profile 14
Task 2: LEFT OUTER JOIN — Missing Records 15
6. Subqueries 16
Task 1: Single-Row Subquery 16
Task 2: Multi-Row Subquery with IN / ANY / ALL 17
7. Views 19
Task 1: Simple Read-Only View 19
Task 2: Payroll Dashboard View 20
8. Stored Procedures & Functions 21
Task 1: Procedure — Add New Employee 21
Task 2: Function — Calculate Net Salary 23
9. Triggers 24
Task 1: BEFORE INSERT — Auto-Assign emp_ID 24
Task 2: AFTER INSERT — Welcome Log 25
10. Oracle Scheduler Jobs 27
Task 1: Simple One-Time Scheduler Job 27
Task 2: Recurring Job — Daily Leave Report 28

1. ERD Design & Mapping

Task 1: Draw the Full ERD



Task 2: Map the ERD to a Relational Schema



SALARY_BONUS.job_ID references JOB_DEPARTMENT(job_ID)

EMPLOYEE.job_ID references JOB_DEPARTMENT(job_ID

EMPLOYEE.salary_ID references SALARY_BONUS(salary_ID)

QUALIFICATION.emp_ID references EMPLOYEE(emp_ID)

QUALIFICATION_REQUIREMENT.qual_ID references QUALIFICATION(qual_ID)

LEAVE.emp_ID references EMPLOYEE(emp_ID)

PAYROLL.emp_ID references EMPLOYEE(emp_ID)

PAYROLL.job_ID references JOB_DEPARTMENT(job_ID)

PAYROLL.salary_ID references SALARY_BONUS(salary_ID)

PAYROLL.leave_ID references LEAVE(leave_ID)

Task 3: Write the Full DDL CREATE Statements

```
-- SEQUENCE to auto-generate primary key
CREATE SEQUENCE JOB_DEPARTMENT_SEQ
START WITH 1
INCREMENT BY 1;

-- JOB_DEPARTMENT
CREATE TABLE JOB_DEPARTMENT (
    job_ID NUMBER PRIMARY KEY,
    job_dept VARCHAR2(100) NOT NULL,
    name VARCHAR2(100) NOT NULL,
    description VARCHAR2(500),
    salary_range VARCHAR2(100) NOT NULL
);
```

```
-- SEQUENCE to auto-generate primary key
CREATE SEQUENCE SALARY_BONUS_SEQ
START WITH 1
INCREMENT BY 1;

-- SALARY_BONUS
CREATE TABLE SALARY_BONUS (
    salary_ID NUMBER PRIMARY KEY,
    amount NUMBER(12,2) NOT NULL,
    annual NUMBER(12,2) NOT NULL,
    bonus NUMBER(12,2) DEFAULT 0 NOT NULL,

    job_ID NUMBER NOT NULL,
    FOREIGN KEY (job_ID) REFERENCES JOB_DEPARTMENT(job_ID),

    CHECK (amount > 0)
);
```

```sql
-- SEQUENCE to auto-generate primary key
CREATE SEQUENCE EMPLOYEE_SEQ
START WITH 1
INCREMENT BY 1;

-- EMPLOYEE
CREATE TABLE EMPLOYEE (
    emp_ID NUMBER PRIMARY KEY,
    fname VARCHAR2(50) NOT NULL,
    lname VARCHAR2(50) NOT NULL,
    gender CHAR(1) NOT NULL,
    age NUMBER(3) NOT NULL,
    contact_add VARCHAR2(250),
    emp_email VARCHAR2(150) NOT NULL,
    emp_pass VARCHAR2(255) NOT NULL,

    job_ID NUMBER NOT NULL,
    salary_ID NUMBER NOT NULL,
    FOREIGN KEY (job_ID) REFERENCES JOB_DEPARTMENT(job_ID),
    FOREIGN KEY (salary_ID) REFERENCES SALARY_BONUS(salary_ID),

    CHECK (gender IN ('M','F'))
);

-- SEQUENCE to auto-generate primary key
CREATE SEQUENCE QUALIFICATION_SEQ
START WITH 1
INCREMENT BY 1;

-- QUALIFICATION
CREATE TABLE QUALIFICATION (
    qual_ID NUMBER PRIMARY KEY,
    position VARCHAR2(100) NOT NULL,
    date_in DATE NOT NULL,

    emp_ID NUMBER NOT NULL,
    FOREIGN KEY (emp_ID) REFERENCES EMPLOYEE(emp_ID)
);

-- QUALIFICATION_REQUIREMENT
-- This table is for Multi-Valued Attribute: requirements
CREATE TABLE QUALIFICATION_REQUIREMENT (
    qual_ID NUMBER,
    requirement VARCHAR2(200),

    FOREIGN KEY (qual_ID) REFERENCES QUALIFICATION(qual_ID),
    PRIMARY KEY (qual_ID, requirement)
);

-- SEQUENCE to auto-generate primary key
CREATE SEQUENCE LEAVE_SEQ
START WITH 1
INCREMENT BY 1;

-- LEAVE_RECORD
CREATE TABLE LEAVE_RECORD (
    leave_ID NUMBER PRIMARY KEY,
    leave_date DATE NOT NULL,
    reason VARCHAR2(250) NOT NULL,

    emp_ID NUMBER NOT NULL,
    FOREIGN KEY (emp_ID) REFERENCES EMPLOYEE(emp_ID)
);
```

```
-- SEQUENCE to auto-generate primary key
CREATE SEQUENCE PAYROLL_SEQ
START WITH 1
INCREMENT BY 1;

-- PAYROLL
CREATE TABLE PAYROLL (
    payroll_ID NUMBER PRIMARY KEY,
    payroll_date DATE NOT NULL,
    report VARCHAR2(500),
    total_amount NUMBER(12,2) NOT NULL,

    emp_ID NUMBER NOT NULL,
    job_ID NUMBER NOT NULL,
    salary_ID NUMBER NOT NULL,
    leave_ID NUMBER,
    FOREIGN KEY (emp_ID) REFERENCES EMPLOYEE(emp_ID),
    FOREIGN KEY (job_ID) REFERENCES JOB_DEPARTMENT(job_ID),
    FOREIGN KEY (salary_ID) REFERENCES SALARY_BONUS(salary_ID),
    FOREIGN KEY (leave_ID) REFERENCES LEAVE_RECORD(leave_ID),

    CHECK (total_amount > 0)
);
```

2. Normalization

Task 1: First Normal Form (1NF) Analysis

- **Check EMS schema against 1NF**

  The EMS schema satisfies First Normal Form because all attributes contain atomic values, and repeating groups are removed into separate tables.

Task 2: Second Normal Form (2NF) Analysis

- **Check EMS schema against 2NF**

  In my EMS schema, most tables have a single-column primary key, so they do not have partial dependency problems.

  The only table with a composite primary key is:

```
CREATE TABLE QUALIFICATION_REQUIREMENT (
    qual_ID NUMBER,
    requirement VARCHAR2(200),
    FOREIGN KEY (qual_ID) REFERENCES QUALIFICATION(qual_ID),
    PRIMARY KEY (qual_ID, requirement)
);
```

  This table does not have any partial dependencies because it has no non-key attributes. Both columns are part of the composite primary key.

  Therefore, the EMS schema satisfies 2NF because there are no non-key attributes that depend on only part of a composite primary key.

Task 3: Third Normal Form (3NF) Analysis

- **Check EMS schema against 2NF**

  The EMS schema is checked against Third Normal Form (3NF). A table is in 3NF if it is already in 2NF and has no transitive dependency. This means that non-key attributes must depend only on the primary key, not on another non-key attribute.

  1. JOB_DEPARTMENT

  All attributes depend directly on job_ID. There is no non-key attribute that determines another non-key attribute.

  Result:

  No transitive dependency exists.

  2. SALARY_BONUS

  The attributes amount, annual, bonus, and job_ID depend directly on salary_ID. The department information is not stored here; only job_ID is stored as a foreign key.

  Result:

  No transitive dependency exists.

  3. EMPLOYEE

  All employee details depend directly on emp_ID. Department details are not stored in EMPLOYEE; only job_ID is stored as a foreign key. Salary details are also not stored in EMPLOYEE; only salary_ID is stored as a foreign key.

  Result:

  No transitive dependency exists.

  4. QUALIFICATION

  The attributes position, date_in, and emp_ID depend directly on qual_ID. Employee details are not stored in this table.

  Result:

  No transitive dependency exists.

  5. QUALIFICATION_REQUIREMENT

  This table has a composite primary key. Both columns form the primary key, and there are no non-key attributes.

  Result:

  No transitive dependency exists.

6. LEAVE_RECORD

The attributes leave_date, reason, and emp_ID depend directly on leave_ID. Employee details are not stored in this table.

Result:

No transitive dependency exists.

7. PAYROLL

All payroll attributes depend directly on payroll_ID. Employee, department, salary, and leave details are not repeated in this table; only their foreign keys are stored.

Result:

No transitive dependency exists.

- Final Statement

  The complete EMS schema is in Third Normal Form (3NF) because all tables are already in 1NF and 2NF, and no table contains transitive dependencies. All non-key attributes depend only on the primary key, the whole primary key, and nothing but the primary key. Therefore, the EMS schema satisfies 3NF.

3. SQL DML — INSERT / UPDATE / DELETE / SELECT

Task 1: Populate All Tables with Seed Data

```
-- 1) JOB_DEPARTMENT: 5 departments
INSERT INTO JOB_DEPARTMENT VALUES (JOB_DEPARTMENT_SEQ.NEXTVAL, 'IT', 'Engineering', 'Software development department', '3000-8000');
INSERT INTO JOB_DEPARTMENT VALUES (JOB_DEPARTMENT_SEQ.NEXTVAL, 'HR', 'Human Resources', 'Employee management department', '2500-6000');
INSERT INTO JOB_DEPARTMENT VALUES (JOB_DEPARTMENT_SEQ.NEXTVAL, 'FIN', 'Finance', 'Payroll and accounting department', '3000-9000');
INSERT INTO JOB_DEPARTMENT VALUES (JOB_DEPARTMENT_SEQ.NEXTVAL, 'MKT', 'Marketing', 'Marketing and sales department', '2500-7000');
INSERT INTO JOB_DEPARTMENT VALUES (JOB_DEPARTMENT_SEQ.NEXTVAL, 'OPS', 'Operations', 'Daily operations department', '2000-6500');

-- 2) SALARY_BONUS: 5 records
INSERT INTO SALARY_BONUS VALUES (SALARY_BONUS_SEQ.NEXTVAL, 4500, 54000, 500, 1);
INSERT INTO SALARY_BONUS VALUES (SALARY_BONUS_SEQ.NEXTVAL, 3200, 38400, 300, 2);
INSERT INTO SALARY_BONUS VALUES (SALARY_BONUS_SEQ.NEXTVAL, 6000, 72000, 700, 3);
INSERT INTO SALARY_BONUS VALUES (SALARY_BONUS_SEQ.NEXTVAL, 3800, 45600, 400, 4);
INSERT INTO SALARY_BONUS VALUES (SALARY_BONUS_SEQ.NEXTVAL, 2900, 34800, 250, 5);

-- 3) EMPLOYEE: 10 employees
INSERT INTO EMPLOYEE VALUES (EMPLOYEE_SEQ.NEXTVAL, 'Ali', 'Al-Balushi', 'M', 28, 'Muscat', 'ali@gmail.com', 'pass123', 1, 1);
INSERT INTO EMPLOYEE VALUES (EMPLOYEE_SEQ.NEXTVAL, 'Sara', 'Al-Harthy', 'F', 32, 'Sohar', 'sara@gmail.com', 'pass123', 2, 2);
INSERT INTO EMPLOYEE VALUES (EMPLOYEE_SEQ.NEXTVAL, 'Ahmed', 'Al-Rawahi', 'M', 41, 'Nizwa', 'ahmed@gmail.com', 'pass123', 3, 3);
INSERT INTO EMPLOYEE VALUES (EMPLOYEE_SEQ.NEXTVAL, 'Maha', 'Al-Siyabi', 'F', 26, 'Muscat', 'maha@gmail.com', 'pass123', 1, 1);
INSERT INTO EMPLOYEE VALUES (EMPLOYEE_SEQ.NEXTVAL, 'Khalid', 'Al-Abri', 'M', 35, 'Sur', 'khalid@gmail.com', 'pass123', 4, 4);
INSERT INTO EMPLOYEE VALUES (EMPLOYEE_SEQ.NEXTVAL, 'Noor', 'Al-Kindi', 'F', 29, 'Ibri', 'noor@gmail.com', 'pass123', 5, 5);
INSERT INTO EMPLOYEE VALUES (EMPLOYEE_SEQ.NEXTVAL, 'Hamad', 'Al-Maawali', 'M', 38, 'Muscat', 'hamad@gmail.com', 'pass123', 3, 3);
INSERT INTO EMPLOYEE VALUES (EMPLOYEE_SEQ.NEXTVAL, 'Aisha', 'Al-Farsi', 'F', 24, 'Salalah', 'aisha@gmail.com', 'pass123', 2, 2);
INSERT INTO EMPLOYEE VALUES (EMPLOYEE_SEQ.NEXTVAL, 'Saeed', 'Al-Nabhani', 'M', 45, 'Muscat', 'saeed@gmail.com', 'pass123', 4, 4);
INSERT INTO EMPLOYEE VALUES (EMPLOYEE_SEQ.NEXTVAL, 'Fatma', 'Al-Amri', 'F', 30, 'Sohar', 'fatma@gmail.com', 'pass123', 5, 5);
```

```
-- 4) QUALIFICATION: 5 records
INSERT INTO QUALIFICATION VALUES (QUALIFICATION_SEQ.NEXTVAL, 'Software Developer', DATE '2023-01-15', 1);
INSERT INTO QUALIFICATION VALUES (QUALIFICATION_SEQ.NEXTVAL, 'HR Officer', DATE '2022-05-20', 2);
INSERT INTO QUALIFICATION VALUES (QUALIFICATION_SEQ.NEXTVAL, 'Accountant', DATE '2021-03-10', 3);
INSERT INTO QUALIFICATION VALUES (QUALIFICATION_SEQ.NEXTVAL, 'Marketing Specialist', DATE '2023-07-01', 5);
INSERT INTO QUALIFICATION VALUES (QUALIFICATION_SEQ.NEXTVAL, 'Operations Officer', DATE '2022-11-12', 6);

-- Multi-valued requirements
INSERT INTO QUALIFICATION_REQUIREMENT VALUES (1, 'SQL');
INSERT INTO QUALIFICATION_REQUIREMENT VALUES (1, 'Oracle');
INSERT INTO QUALIFICATION_REQUIREMENT VALUES (2, 'Communication Skills');
INSERT INTO QUALIFICATION_REQUIREMENT VALUES (3, 'Accounting Knowledge');
INSERT INTO QUALIFICATION_REQUIREMENT VALUES (4, 'Marketing Strategy');
INSERT INTO QUALIFICATION_REQUIREMENT VALUES (5, 'Operations Management');

-- 5) LEAVE_RECORD: 5 records
INSERT INTO LEAVE_RECORD VALUES (LEAVE_SEQ.NEXTVAL, DATE '2024-01-10', 'Sick leave', 1);
INSERT INTO LEAVE_RECORD VALUES (LEAVE_SEQ.NEXTVAL, DATE '2024-02-15', 'Annual leave', 2);
INSERT INTO LEAVE_RECORD VALUES (LEAVE_SEQ.NEXTVAL, DATE '2024-03-20', 'Emergency leave', 3);
INSERT INTO LEAVE_RECORD VALUES (LEAVE_SEQ.NEXTVAL, DATE '2024-04-05', 'Sick leave due to flu', 5);
INSERT INTO LEAVE_RECORD VALUES (LEAVE_SEQ.NEXTVAL, DATE '2024-05-12', 'Family leave', 6);

-- 6) PAYROLL: 8 records
INSERT INTO PAYROLL VALUES (PAYROLL_SEQ.NEXTVAL, DATE '2024-01-31', 'January payroll', 5000, 1, 1, 1, 1);
INSERT INTO PAYROLL VALUES (PAYROLL_SEQ.NEXTVAL, DATE '2024-01-31', 'January payroll', 3500, 2, 2, 2, 2);
INSERT INTO PAYROLL VALUES (PAYROLL_SEQ.NEXTVAL, DATE '2024-01-31', 'January payroll', 6700, 3, 3, 3, 3);
INSERT INTO PAYROLL VALUES (PAYROLL_SEQ.NEXTVAL, DATE '2024-02-29', 'February payroll', 5000, 4, 1, 1, NULL);
INSERT INTO PAYROLL VALUES (PAYROLL_SEQ.NEXTVAL, DATE '2024-02-29', 'February payroll', 4200, 5, 4, 4, 4);
INSERT INTO PAYROLL VALUES (PAYROLL_SEQ.NEXTVAL, DATE '2024-02-29', 'February payroll', 3150, 6, 5, 5, 5);
INSERT INTO PAYROLL VALUES (PAYROLL_SEQ.NEXTVAL, DATE '2024-03-31', 'March payroll', 6700, 7, 3, 3, NULL);
INSERT INTO PAYROLL VALUES (PAYROLL_SEQ.NEXTVAL, DATE '2024-03-31', 'March payroll', 3500, 8, 2, 2, NULL);
```

Task 2: Conditional SELECT Queries

a) Employees whose age is between 25 and 40

```
SELECT emp_ID,
       fname,
       lname,
       age
FROM EMPLOYEE
WHERE age BETWEEN 25 AND 40
ORDER BY lname ASC;
```

EMP_ID	FNAME	LNAME	AGE
5	Khalid	Al-Abri	35
10	Fatma	Al-Amri	30
1	Ali	Al-Balushi	28
2	Sara	Al-Harthy	32
6	Noor	Al-Kindi	29
7	Hamad	Al-Maawali	38
4	Maha	Al-Siyabi	26

b) Payroll records where total_amount exceeds 5000

```
SELECT p.payroll_ID,
       e.fname || ' ' || e.lname AS employee_name,
       jd.name AS department_name,
       p.total_amount
FROM PAYROLL p
JOIN EMPLOYEE e
ON p.emp_ID = e.emp_ID
JOIN JOB_DEPARTMENT jd
ON p.job_ID = jd.job_ID
WHERE p.total_amount > 5000;
```

PAYROLL_ID	EMPLOYEE_NAME	DEPARTMENT_NAME	TOTAL_AMOUNT
3	Ahmed Al-Rawahi	Finance	6700
7	Hamad Al-Maawali	Finance	6700

c) Employees who have taken leave with reason containing 'sick'

```
SELECT e.emp_ID,
       e.fname || ' ' || e.lname AS employee_name,
       l.leave_date,
       l.reason
FROM EMPLOYEE e
JOIN LEAVE_RECORD l
ON e.emp_ID = l.emp_ID
WHERE LOWER(l.reason) LIKE '%sick%';
```

EMP_ID	EMPLOYEE_NAME	LEAVE_DATE	REASON
1	Ali Al-Balushi	10-JAN-24	Sick leave
5	Khalid Al-Abri	05-APR-24	Sick leave due to flu

d) Departments that have no employees assigned

```
SELECT jd.job_ID,
       jd.name AS department_name
FROM JOB_DEPARTMENT jd
WHERE NOT EXISTS (
    SELECT 1
    FROM EMPLOYEE e
    WHERE e.job_ID = jd.job_ID
);
```

JOB_ID	DEPARTMENT_NAME
7	Engineering
11	Operations
8	Human Resources
10	Marketing
9	Finance

4. Aggregation Functions

Task 1: Basic Aggregation

a) Total number of employees in each department

```
SELECT jd.job_ID,
       jd.name AS department_name,
       COUNT(e.emp_ID) AS total_employees
FROM JOB_DEPARTMENT jd
LEFT JOIN EMPLOYEE e
ON jd.job_ID = e.job_ID
GROUP BY jd.job_ID, jd.name
ORDER BY id.name;
```

JOB_ID	DEPARTMENT_NAME	TOTAL_EMPLOYEES
1	Engineering	2
7	Engineering	0
3	Finance	2
9	Finance	0
2	Human Resources	2
8	Human Resources	0
4	Marketing	2
10	Marketing	0
5	Operations	2
11	Operations	0

b) Minimum, maximum, and average salary across all salary records

```
SELECT MIN(amount) AS minimum_salary,
       MAX(amount) AS maximum_salary,
       AVG(amount) AS average_salary
FROM SALARY_BONUS;
```

MINIMUM_SALARY	MAXIMUM_SALARY	AVERAGE_SALARY
2900	6000	4080

c) Total bonus paid out across the entire company

```
SELECT SUM(bonus) AS total_bonus_paid
FROM SALARY_BONUS;
```

TOTAL_BONUS_PAID
2150

Task 2: GROUP BY with HAVING

a) Departments where the average employee age exceeds 30

```
SELECT jd.job_ID,
       jd.name AS department_name,
       AVG(e.age) AS average_age
FROM JOB_DEPARTMENT jd
JOIN EMPLOYEE e
ON jd.job_ID = e.job_ID
GROUP BY jd.job_ID, jd.name
HAVING AVG(e.age) > 30
ORDER BY average_age DESC;
```

JOB_ID	DEPARTMENT_NAME	AVERAGE_AGE
4	Marketing	40
3	Finance	39.5

b) Job titles where more than 2 employees share that qualification position

```
SELECT q.position AS job_title,
       COUNT(q.emp_ID) AS employee_count
FROM QUALIFICATION q
GROUP BY q.position
HAVING COUNT(q.emp_ID) > 2
ORDER BY employee_count DESC;
```

JOB_TITLE	EMPLOYE...

c) Months where the total payroll amount exceeds 20,000

```
SELECT TO_CHAR(payroll_date, 'YYYY-MM') AS payroll_month,
       SUM(total_amount) AS total_monthly_payroll
FROM PAYROLL
GROUP BY TO_CHAR(payroll_date, 'YYYY-MM')
HAVING SUM(total_amount) > 20000
ORDER BY payroll_month;
```

PAYROLL...	TOTAL_M...

5. Joins

Task 1: INNER JOIN — Employee Full Profile

```
SELECT e.emp_ID,
       e.fname || ' ' || e.lname AS full_name,
       jd.name AS department_name,
       q.position AS job_title,
       sb.amount AS salary_amount,
       MAX(l.leave_date) AS latest_leave_date
FROM EMPLOYEE e
INNER JOIN PAYROLL p
    ON e.emp_ID = p.emp_ID
INNER JOIN QUALIFICATION q
    ON e.emp_ID = q.emp_ID
INNER JOIN JOB_DEPARTMENT jd
    ON e.job_ID = jd.job_ID
INNER JOIN SALARY_BONUS sb
    ON e.salary_ID = sb.salary_ID
LEFT JOIN LEAVE_RECORD l
    ON e.emp_ID = l.emp_ID
GROUP BY e.emp_ID,
         e.fname,
         e.lname,
         jd.name,
         q.position,
         sb.amount
ORDER BY e.emp_ID;
```

EMP_ID	FULL_NAME	DEPARTMENT_NAME	JOB_TITLE	SALARY_AMOUNT	LATEST_LEAVE_DATE
1	Ali Al-Balushi	Engineering	Software Developer	4500	10-JAN-24
2	Sara Al-Harthy	Human Resources	HR Officer	3200	15-FEB-24
3	Ahmed Al-Rawahi	Finance	Accountant	6000	20-MAR-24
5	Khalid Al-Abri	Marketing	Marketing Specialist	3800	05-APR-24
6	Noor Al-Kindi	Operations	Operations Officer	2900	12-MAY-24

Task 2: LEFT OUTER JOIN — Missing Records

a) Employees Who Never Took Leave

```
SELECT e.emp_ID,
       e.fname,
       e.lname
FROM EMPLOYEE e
LEFT JOIN LEAVE_RECORD l
    ON e.emp_ID = l.emp_ID
WHERE l.leave_ID IS NULL;
```

EMP_ID	FNAME	LNAME
7	Hamad	Al-Maawali
8	Aisha	Al-Farsi
4	Maha	Al-Siyabi
10	Fatma	Al-Amri
9	Saeed	Al-Nabhani

b) Departments With No Salary/Bonus Records

```
SELECT jd.job_ID,
       jd.name AS department_name
FROM JOB_DEPARTMENT jd
LEFT JOIN SALARY_BONUS sb
    ON jd.job_ID = sb.job_ID
WHERE sb.salary_ID IS NULL;
```

JOB_ID	DEPARTMENT_NAME
7	Engineering
11	Operations
8	Human Resources
10	Marketing
9	Finance

6. Subqueries

Task 1: Single-Row Subquery

a) Employees whose salary is greater than the company average salary

```
SELECT e.emp_ID,
       e.fname,
       e.lname,
       sb.amount
FROM EMPLOYEE e
JOIN SALARY_BONUS sb
ON e.salary_ID = sb.salary_ID
WHERE sb.amount >
(
    SELECT AVG(amount)
    FROM SALARY_BONUS
);
```

EMP_ID	FNAME	LNAME	AMOUNT
1	Ali	Al-Balushi	4500
3	Ahmed	Al-Rawahi	6000
4	Maha	Al-Siyabi	4500
7	Hamad	Al-Maawali	6000

b) Department with the highest total payroll amount

```
SELECT jd.job_ID,
       jd.name,
       SUM(p.total_amount) AS total_payroll
FROM JOB_DEPARTMENT jd
JOIN PAYROLL p
ON jd.job_ID = p.job_ID
GROUP BY jd.job_ID, jd.name
HAVING SUM(p.total_amount) =
(
    SELECT MAX(total_payroll)
    FROM
    (
        SELECT SUM(total_amount) AS total_payroll
        FROM PAYROLL
        GROUP BY job_ID
    )
);
```

JOB_ID	NAME	TOTAL_PAYROLL
3	Finance	13400

Task 2: Multi-Row Subquery with IN / ANY / ALL

a) Employees who work in departments that have at least one bonus greater than 500 (Using IN)

```
SELECT emp_ID,
       fname,
       lname
FROM EMPLOYEE
WHERE job_ID IN
(
    SELECT job_ID
    FROM SALARY_BONUS
    WHERE bonus > 500
);
```

EMP_ID	FNAME	LNAME
3	Ahmed	Al-Rawahi
7	Hamad	Al-Maawali

b) Employees whose salary is greater than ALL salaries in the Maintenance department (Using ALL)

```
SELECT e.emp_ID,
       e.fname,
       e.lname,
       sb.amount
FROM EMPLOYEE e
JOIN SALARY_BONUS sb
ON e.salary_ID = sb.salary_ID
WHERE sb.amount > ALL
(
    SELECT sb2.amount
    FROM EMPLOYEE e2
    JOIN SALARY_BONUS sb2
    ON e2.salary_ID = sb2.salary_ID
    JOIN JOB_DEPARTMENT jd
    ON e2.job_ID = jd.job_ID
    WHERE jd.name = 'Maintenance'
);
```

EMP_ID	FNAME	LNAME	AMOUNT
1	Ali	Al-Balushi	4500
2	Sara	Al-Harthy	3200
3	Ahmed	Al-Rawahi	6000
4	Maha	Al-Siyabi	4500
5	Khalid	Al-Abri	3800
6	Noor	Al-Kindi	2900
7	Hamad	Al-Maawali	6000
8	Aisha	Al-Farsi	3200
9	Saeed	Al-Nabhani	3800
10	Fatma	Al-Amri	2900

c) Employees whose salary is greater than ANY salary in the HR department (Using ANY)

```
SELECT e.emp_ID,
       e.fname,
       e.lname,
       sb.amount
FROM EMPLOYEE e
JOIN SALARY_BONUS sb
ON e.salary_ID = sb.salary_ID
WHERE sb.amount > ANY
(
    SELECT sb2.amount
    FROM EMPLOYEE e2
    JOIN SALARY_BONUS sb2
    ON e2.salary_ID = sb2.salary_ID
    JOIN JOB_DEPARTMENT jd
    ON e2.job_ID = jd.job_ID
    WHERE jd.name = 'Human Resources'
);
```

EMP_ID	FNAME	LNAME	AMOUNT
1	Ali	Al-Balushi	4500
3	Ahmed	Al-Rawahi	6000
4	Maha	Al-Siyabi	4500
5	Khalid	Al-Abri	3800
7	Hamad	Al-Maawali	6000
9	Saeed	Al-Nabhani	3800

7. Views

Task 1: Simple Read-Only View

a) Create View

```
CREATE OR REPLACE VIEW VW_EMPLOYEE_SUMMARY AS
SELECT e.emp_ID,
       e.fname || ' ' || e.lname AS full_name,
       e.gender,
       e.age,
       jd.name AS department_name,
       q.position AS job_title
FROM EMPLOYEE e
JOIN JOB_DEPARTMENT jd
ON e.job_ID = jd.job_ID
JOIN QUALIFICATION q
ON e.emp_ID = q.emp_ID;
```

b) List all female employees over 30

```
SELECT *
FROM VW_EMPLOYEE_SUMMARY
WHERE gender = 'F'
AND age > 30;
```

EMP_ID	FULL_NAME	GENDER	AGE	DEPARTMENT_NAME	JOB_TITLE
2	Sara Al-Harthy	F	32	Human Resources	HR Officer

c) Try to INSERT through the view

```
INSERT INTO VW_EMPLOYEE_SUMMARY
VALUES (
    100,
    'Test User',
    'F',
    35,
    'Human Resources',
    'HR Officer'
);
```

Expected Oracle Error:

```
Error report -
SQL Error: ORA-01779: cannot modify a column which maps to a non key-preserved table
```

Explanation:

The view is based on multiple joined tables (EMPLOYEE, JOB_DEPARTMENT, QUALIFICATION); therefore, Oracle treats it as a non-updatable view.

Task 2: Payroll Dashboard View

a) Create View

```
CREATE OR REPLACE VIEW VW_PAYROLL_DASHBOARD AS
SELECT p.payroll_ID,
       e.fname || ' ' || e.lname AS employee_name,
       jd.name AS department_name,
       sb.amount AS salary_amount,
       sb.bonus,
       l.reason AS leave_reason,
       p.payroll_date,
       p.total_amount
FROM PAYROLL p
JOIN EMPLOYEE e
ON p.emp_ID = e.emp_ID
JOIN JOB_DEPARTMENT jd
ON p.job_ID = jd.job_ID
JOIN SALARY_BONUS sb
ON p.salary_ID = sb.salary_ID
LEFT JOIN LEAVE_RECORD l
ON p.leave_ID = l.leave_ID;
```

b) Top 5 Payroll Records by Total Amount

```
SELECT *
FROM VW_PAYROLL_DASHBOARD
ORDER BY total_amount DESC
FETCH FIRST 5 ROWS ONLY;
```

PAYROLL_ID	EMPLOYEE_NAME	DEPARTMENT_NAME	SALARY_AMOUNT	BONUS	LEAVE_REASON	PAYROLL_DATE	TOTAL_AMOUNT
3	Ahmed Al-Rawahi	Finance	6000	700	Emergency leave	31-JAN-24	6700
7	Hamad Al-Maawali	Finance	6000	700	(null)	31-MAR-24	6700
1	Ali Al-Balushi	Engineering	4500	500	Sick leave	31-JAN-24	5000
4	Maha Al-Siyabi	Engineering	4500	500	(null)	29-FEB-24	5000
5	Khalid Al-Abri	Marketing	3800	400	Sick leave due to flu	29-FEB-24	4200

8. Stored Procedures & Functions

Task 1: Procedure — Add New Employee

a) Create Procedure

```
CREATE OR REPLACE PROCEDURE SP_ADD_EMPLOYEE (
    p_fname         VARCHAR2,
    p_lname         VARCHAR2,
    p_gender        CHAR,
    p_age           NUMBER,
    p_contact_add   VARCHAR2,
    p_emp_email     VARCHAR2,
    p_emp_pass      VARCHAR2,
    p_job_ID        NUMBER,
    p_salary_ID     NUMBER
)
IS
    v_count NUMBER;
BEGIN

    SELECT COUNT(*)
    INTO v_count
    FROM EMPLOYEE
    WHERE emp_email = p_emp_email;

    IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Email already exists.'
        );
    END IF;

    INSERT INTO EMPLOYEE (
        emp_ID,
        fname,
        lname,
        gender,
        age,
        contact_add,
        emp_email,
        emp_pass,
        job_ID,
        salary_ID
    )
    VALUES (
        EMPLOYEE_SEQ.NEXTVAL,
        p_fname,
        p_lname,
        p_gender,
        p_age,
        p_contact_add,
        p_emp_email,
        p_emp_pass,
        p_job_ID,
        p_salary_ID
    );

    DBMS_OUTPUT.PUT_LINE('Employee added successfully.');

    COMMIT;

END;
/
```

b) First Test (Success)

```
BEGIN
    SP_ADD_EMPLOYEE(
        'Omar',
        'Al-Busaidi',
        'M',
        27,
        'Muscat',
        'omar@gmail.com',
        '123456',
        1,
        1
    );
END;
/
```

```
Employee added successfully.
```

c) Second Test (Duplicate Email)

```
BEGIN
    SP_ADD_EMPLOYEE(
        'Omar',
        'Al-Busaidi',
        'M',
        27,
        'Muscat',
        'omar@gmail.com',
        '123456',
        1,
        1
    );
END;
/
```

```
ORA-20001: Email already exists.
```

Task 2: Function — Calculate Net Salary

a) Create Function

```
CREATE OR REPLACE FUNCTION FN_NET_SALARY (
    p_emp_ID NUMBER
)
RETURN NUMBER
IS
    v_net_salary NUMBER;
BEGIN

    SELECT sb.amount + sb.bonus
    INTO v_net_salary
    FROM EMPLOYEE e
    JOIN SALARY_BONUS sb
    ON e.salary_ID = sb.salary_ID
    WHERE e.emp_ID = p_emp_ID;

    RETURN v_net_salary;

END;
/
```

b) Use Function in SELECT Query

```
SELECT emp_ID,
       fname,
       lname,
       FN_NET_SALARY(emp_ID) AS net_salary
FROM EMPLOYEE
ORDER BY FN_NET_SALARY(emp_ID) DESC;
```

EMP_ID	FNAME	LNAME	NET_SALARY
7	Hamad	Al-Maawali	6700
3	Ahmed	Al-Rawahi	6700
12	Omar	Al-Busaidi	5000
1	Ali	Al-Balushi	5000
4	Maha	Al-Siyabi	5000
11	Mohammed	Al-Harthi	5000
5	Khalid	Al-Abri	4200
9	Saeed	Al-Nabhani	4200
8	Aisha	Al-Farsi	3500
2	Sara	Al-Harthy	3500
10	Fatma	Al-Amri	3150
6	Noor	Al-Kindi	3150

9. Triggers

Task 1: BEFORE INSERT — Auto-Assign emp_ID

a) Create Trigger

```
CREATE OR REPLACE TRIGGER TRG_EMP_ID
BEFORE INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
    IF :NEW.emp_ID IS NULL THEN
        :NEW.emp_ID := EMPLOYEE_SEQ.NEXTVAL;
    END IF;
END;
/
```

b) Test Trigger

```
INSERT INTO EMPLOYEE (
    fname,
    lname,
    gender,
    age,
    contact_add,
    emp_email,
    emp_pass,
    job_ID,
    salary_ID
)
VALUES (
    'Ali',
    'Al-Harthi',
    'M',
    30,
    'Muscat',
    'ali_test@gmail.com',
    '123456',
    1,
    1
);
```

c) Verify

```
SELECT emp_ID,
       fname,
       lname
FROM EMPLOYEE
WHERE emp_email = 'ali_test@gmail.com';
```

EMP_ID	FNAME	LNAME
13	Ali	Al-Harthi

Task 2: AFTER INSERT — Welcome Log

a) Create Log Table

```
CREATE TABLE EMPLOYEE_LOG (
    log_ID NUMBER PRIMARY KEY,
    emp_ID NUMBER,
    action VARCHAR2(50),
    log_timestamp TIMESTAMP
);
```

b) Create Log Sequence

```
CREATE SEQUENCE EMPLOYEE_LOG_SEQ
START WITH 1
INCREMENT BY 1;
```

c) Create Trigger

```
CREATE OR REPLACE TRIGGER TRG_EMP_WELCOME_LOG
AFTER INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
    INSERT INTO EMPLOYEE_LOG (
        log_ID,
        emp_ID,
        action,
        log_timestamp
    )
    VALUES (
        EMPLOYEE_LOG_SEQ.NEXTVAL,
        :NEW.emp_ID,
        'NEW HIRE',
        SYSTIMESTAMP
    );
END;
/
```

d) Test

Employee 1

```
INSERT INTO EMPLOYEE (
    fname,
    lname,
    gender,
    age,
    contact_add,
    emp_email,
    emp_pass,
    job_ID,
    salary_ID
)
VALUES (
    'Sara',
    'Al-Amri',
    'F',
    28,
    'Sohar',
    'sara_test@gmail.com',
    '123456',
    2,
    2
);
```

Employee 2

```
INSERT INTO EMPLOYEE (
    fname,
    lname,
    gender,
    age,
    contact_add,
    emp_email,
    emp_pass,
    job_ID,
    salary_ID
)
VALUES (
    'Ahmed',
    'Al-Busaidi',
    'M',
    35,
    'Muscat',
    'ahmed_test@gmail.com',
    '123456',
    3,
    3
);
```

e) Verify Log Records

```
SELECT *
FROM EMPLOYEE_LOG
ORDER BY log_ID;
```

LOG_ID	EMP_ID	ACTION	LOG_TIMESTAMP
1	14	NEW HIRE	19-JUN-26 11.43.30.552000000 PM
2	15	NEW HIRE	19-JUN-26 11.44.07.041000000 PM

10. Oracle Scheduler Jobs

Task 1: Simple One-Time Scheduler Job

a) Create Job

```
BEGIN
    DBMS_SCHEDULER.CREATE_JOB (
        job_name   => 'JOB_GREET_EMPLOYEES',
        job_type   => 'PLSQL_BLOCK',
        job_action => '
        BEGIN
            DBMS_OUTPUT.PUT_LINE(''Payroll System Initialized'');

            INSERT INTO EMPLOYEE_LOG (
                log_ID,
                emp_ID,
                action,
                log_timestamp
            )
            VALUES (
                EMPLOYEE_LOG_SEQ.NEXTVAL,
                NULL,
                ''PAYROLL SYSTEM INITIALIZED'',
                SYSTIMESTAMP
            );

            COMMIT;
        END;',
        start_date => SYSTIMESTAMP + INTERVAL '2' MINUTE,
        enabled    => TRUE
    );
END;
/
```

b) Check Job

```
SELECT job_name,
       enabled,
       state
FROM USER_SCHEDULER_JOBS
WHERE job_name = 'JOB_GREET_EMPLOYEES';
```

JOB_NAME	ENABLED	STATE
JOB_GREET_EMPLOYEES	TRUE	SCHEDULED

c) Check Job Log

```
SELECT *
FROM USER_SCHEDULER_JOB_LOG
WHERE job_name = 'JOB_GREET_EMPLOYEES'
ORDER BY log_date DESC;
```

LOG_ID	LOG_DATE	OWNER	JOB_NAME	JOB...	JOB_CLASS	...	STATUS	USE...	CLI...	GLO...	CRE...	CRE...	DES...	DES...	ADDIT..
2944	20-JUN-26 12.06.55.993000000 AM +04:00	CAPSTONEPROJECT	JOB_GREET_EMPLOYEES	(null)	DEFAULT_JOB_CLASS	RUN	SUCCEEDED	(null)	(null)	(null)	(null)	(null)	(null)	(null)	(null)

d) Check EMPLOYEE_LOG

```
SELECT *
FROM EMPLOYEE_LOG
ORDER BY log_ID DESC;
```

LOG_ID	EMP_ID	ACTION	LOG_TIMESTAMP
3	(null)	PAYROLL SYSTEM INITIALIZED	20-JUN-26 12.06.55.910000000 AM
2	15	NEW HIRE	19-JUN-26 11.44.07.041000000 PM
1	14	NEW HIRE	19-JUN-26 11.43.30.552000000 PM

Task 2: Recurring Job — Daily Leave Report

a) Create Job

```
BEGIN
    DBMS_SCHEDULER.CREATE_JOB (
        job_name        => 'JOB_DAILY_LEAVE_REPORT',
        job_type        => 'PLSQL_BLOCK',
        job_action      => '
        DECLARE
            v_leave_count NUMBER;
        BEGIN
            SELECT COUNT(*)
            INTO v_leave_count
            FROM LEAVE_RECORD
            WHERE TRUNC(leave_date) = TRUNC(SYSDATE);
            INSERT INTO EMPLOYEE_LOG (
                log_ID,
                emp_ID,
                action,
                log_timestamp
            )
            VALUES (
                EMPLOYEE_LOG_SEQ.NEXTVAL,
                NULL,
                ''DAILY LEAVE COUNT: '' || v_leave_count,
                SYSTIMESTAMP
            );
            COMMIT;
        END;',
        start_date      => SYSTIMESTAMP,
        repeat_interval => 'FREQ=DAILY;BYHOUR=7;BYMINUTE=0;BYSECOND=0',
        enabled         => TRUE
    );
END;
/
```

b) Show Job Definition

```
SELECT *
FROM USER_SCHEDULER_JOBS
WHERE JOB_NAME = 'JOB_DAILY_LEAVE_REPORT';
```

c) Check Next Run Time

```
SELECT job_name,
       state,
       next_run_date
FROM USER_SCHEDULER_JOBS
WHERE job_name = 'JOB_DAILY_LEAVE_REPORT';
```

JOB_NAME	STATE	NEXT_RUN_DATE
JOB_DAILY_LEAVE_REPORT	SCHEDULED	20-JUN-26 07.00.00.465000000 AM +04:00

d) Check Execution Log

```
SELECT *
FROM USER_SCHEDULER_JOB_LOG
WHERE job_name = 'JOB_DAILY_LEAVE_REPORT'
ORDER BY log_date DESC;
```